Build & Deploy Documentation

Zero-Downtime Linux Updates — Build, Test & Deployment Guide

This document covers how to build, test, and deploy the project components.

Table of Contents

Repository Structure

```
amos2026ss01-zero-downtime-linux-updates/
├── Cargo.toml              — Workspace root manifest
├── Makefile                — Developer setup helpers
├── orchestrator/           — Main agent binary (amos-orchestrator)
│   ├── Cargo.toml
│   ├── config.example.toml
│   └── src/
├── common/                 — Shared library crate (amos-common)
│   │                         includes download manager module and security
verification
│   ├── Cargo.toml
│   └── src/
├── api-mock-server/        — Development mock server binary
│   ├── Cargo.toml
│   └── src/
└── rootc-build/            — Container image build files
    ├── Containerfile
    └── orchestrator.service
```

Prerequisites

For local development

Tool	Version	Purpose
Rust	stable (≥ 1.80)	Build toolchain
`cargo`	(included with Rust)	Package manager & build tool
`git`	≥ 2.x	Version control
`make`	any	Developer setup shortcuts

Optional (for full integration testing):

- `podman` — container runtime
- `bootc` — OS image tooling
- `rpm-ostree` — OSTree management

For building container images

- `podman` or `docker` with OCI image support
- Access to a container registry (GHCR)

Initial Setup

After cloning the repository, run the one-time developer setup:

```
git clone https://github.com/amosproj/amos2026ss01-zero-downtime-linux-updates.git
cd amos2026ss01-zero-downtime-linux-updates
make setup
```

`make setup` configures:

- A **commit message template** (conventional commits format).
- A **Git hook** that automatically appends the DCO sign-off line to commit messages.

> **Note:** All commits must be signed off (`git commit -s`) per the project's Developer Certificate of Origin.

See CONTRIBUTING.md for full contribution guidelines.

Building the Project

Build all workspace crates (debug mode)

```
cargo build
```

Compiled binaries are placed in `target/debug/`:

- `target/debug/amos-orchestrator`
- `target/debug/amos-api-mock-server`

Build in release mode (optimised, for deployment)

```
cargo build --release
```

Compiled binaries are placed in `target/release/`:

- `target/release/amos-orchestrator`
- `target/release/amos-api-mock-server`

Build a specific crate only

```
cargo build -p amos-orchestrator
cargo build -p amos-api-mock-server
```

Running Tests

Run all tests

```
cargo test
```

Run tests for a specific crate

```
cargo test -p amos-orchestrator
cargo test -p amos-common
cargo test -p security-module
```

Notable test coverage

Crate	Tests
`amos-orchestrator`	CLI flag parsing (`--self-check`, `--config`, `--debug`)
`amos-orchestrator`	Config validation (URL scheme, poll interval)
`amos-common`	`CatalogResponse` JSON serialisation/deserialisation
`security-module`	Ed25519 signature verification (valid, invalid, tampered)

Running Locally

0. Start *a* database

The api server requires a database, which by default is expected as a Postgres instance running on `localhost:5432`. A local container instance can be managed like this:

```
cd .devcontainer/

# start the container
podman-compose up -d postgres

# stop the container
podman-compose down postgres

# optional: to delete the data (volume)
podman volume rm devcontainer_postgres_data
```

Alternatively for local development an sqlite database can be used. For that, just define a sqlite connection string via the config or an environment variable when running the server, e.g.: `APP_DATABASE_URL="sqlite://db.sqlite?mode=rwc" cargo run ..."`

1. Start the API mock server

The mock server simulates the Cloud API on `localhost:80`:

```
# Requires port 80 — run with sudo or change to a high port (edit source if
needed)
sudo ./target/debug/amos-api-mock-server
```

> **Note:** Port 80 requires root privileges. If you do not want to use `sudo`, edit `api-mock-server/src/main.rs` to bind to a high port (e.g. `8080`) and rebuild.

Place any binary update artifacts you want to serve in an `assets/` directory next to the binary.

2. Create a config file

```
cp orchestrator/config.example.toml config.toml
```

Edit `config.toml` to point at the mock server:

```
cloud_url = "http://localhost/api/v1"
poll_interval_secs = 10
inventory_path = "./inventory/inventory.json"
```

> **Note:** `cloud_url` accepts both `http://` and `https://` schemas. For local development, pointing directly at the plain-HTTP mock server with `http://localhost` is the simplest setup.

3. Run the Orchestrator

```
./target/debug/amos-orchestrator --config config.toml -d
```

The `-d` flag enables `INFO` level logging so you can follow the update loop.

4. Verify with self-check

```
./target/debug/amos-orchestrator --self-check --config config.toml
```

Exit code 0 means the agent is correctly configured and inventory tooling is available.

Deploying to an Edge Device

1. Copy the release binary

```
cargo build --release
scp target/release/amos-orchestrator user@edge-device:/usr/local/bin/
ssh user@edge-device "chmod +x /usr/local/bin/amos-orchestrator"
```

2. Create the config file on the device

```
ssh user@edge-device "sudo mkdir -p /etc/amos"
scp orchestrator/config.example.toml user@edge-device:/etc/amos/config.toml
# Edit /etc/amos/config.toml on the device with the correct cloud_url
```

3. Install and enable the systemd service

```
scp rootc-build/orchestrator.service user@edge-device:/tmp/
ssh user@edge-device "sudo cp /tmp/orchestrator.service /etc/systemd/system/ && \
  sudo systemctl daemon-reload && \
  sudo systemctl enable --now amos-orchestrator"
```

4. Verify

```
ssh user@edge-device "sudo systemctl status amos-orchestrator"
ssh user@edge-device "sudo journalctl -u amos-orchestrator -n 50"
```

Container / rootc Build

The `rootc-build/` directory contains the files needed to embed the Orchestrator into an OS container image.

Files

File	Description
`rootc-build/Containerfile`	OCI image definition for the root container build
`rootc-build/orchestrator.service`	systemd unit file bundled into the image

Build the container image

```
podman build -f rootc-build/Containerfile -t amos-orchestrator:latest .
```

Push to GHCR

```
podman tag amos-orchestrator:latest ghcr.io/amosproj/amos2026ss01-zero-downtime-
linux-updates-system:latest
podman push ghcr.io/amosproj/amos2026ss01-zero-downtime-linux-updates-
system:latest
```

CI / CD

The project uses GitHub Actions. Workflows are defined under `.github/workflows/` (if present). The typical pipeline:

1. **Build** — `cargo check` on push/PR
2. **Lint** — `cargo clippy -- -D warnings`
3. **Format check** — `cargo fmt --check`
4. **Test** — `cargo test` on all crates
5. **Release build** — `cargo build --release` on tagged push

Changelogs are auto-generated from conventional commits using `git-cliff` (configured in `cliff.toml`).

Environment Variables Reference

> See User Documentation — Configuration for the full environment variable reference.